

MODULE5

Android View

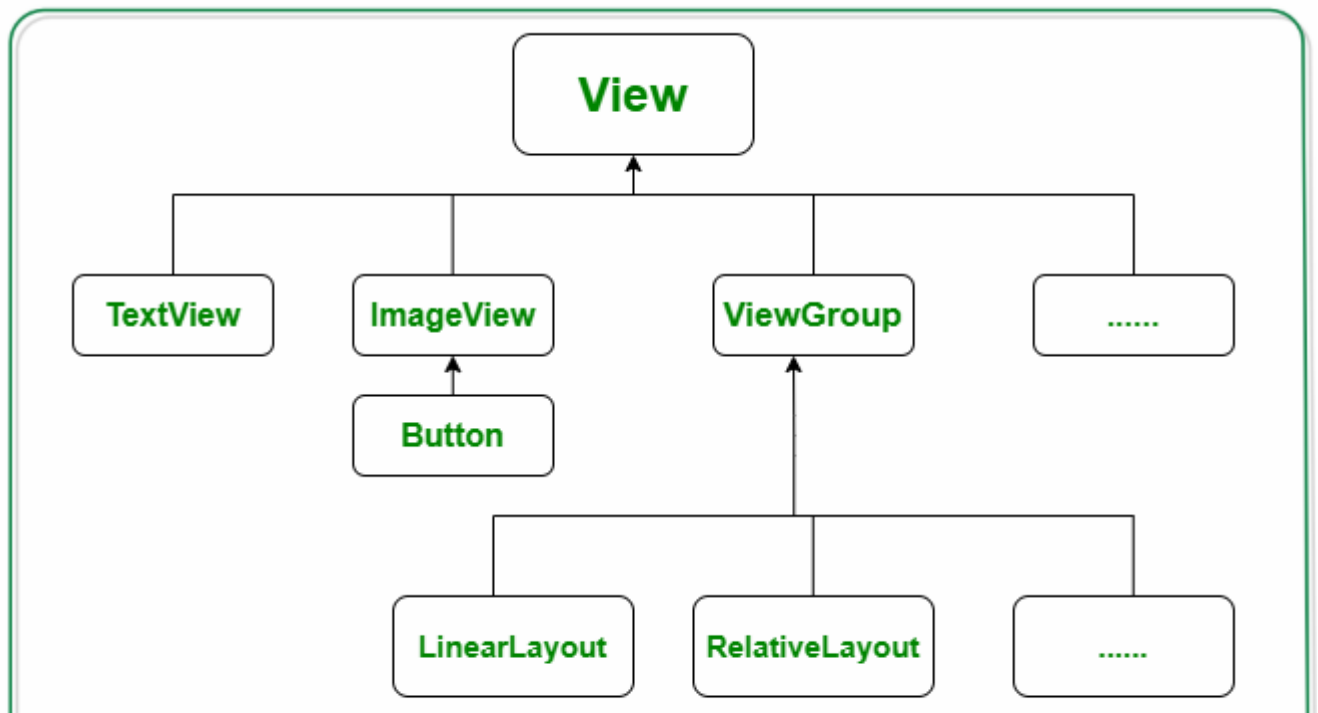
The **View** is a base class for all UI components in android. For example, the **EditText** class is used to accept the input from users in android apps, which is a subclass of **View**.

Following are the some of common **View** subclasses that will be used in android applications.

- [TextView](#)
- [EditText](#)
- [Button](#)
- [CheckBox](#)
- [RadioButton](#)
- [ImageButton](#)
- [Progress Bar](#)
- [Spinner](#)

Android ViewGroup

The **ViewGroup** is a subclass of **View** and it will act as a base class for **layouts** and **layouts parameters**. For example, [Linear Layout](#) is the **ViewGroup** that contains a UI controls like button, textview, etc. and other layouts also.



Difference Table

View	ViewGroup
View is a simple rectangle box that responds to the user's actions.	ViewGroup is the invisible container. It holds View and ViewGroup
View is the SuperClass of All component like TextView, EditText, ListView, etc	ViewGroup is a collection of Views(TextView, EditText, ListView, etc..), somewhat like a container.
A View object is a component of the user interface (UI) like a button or a text box, and it's also called a widget.	A ViewGroup object is a layout, that is, a container of other ViewGroup objects (layouts) and View objects (widgets)
Examples are EditText, Button, CheckBox, etc.	For example, LinearLayout is the ViewGroup that contains Button(View), and other Layouts also.
View refers to the	ViewGroup refers to the android.view.ViewGroup

android.view.View class	class
android.view.View which is the base class of all UI classes.	ViewGroup is the base class for Layouts.

Checkbox

CheckBox is used for adding multiple selections of items from the given set of options. It is generally used in a place where user can select one or more than choices from a given list of choices. For example, selecting hobbies.

Attributes of Checkbox:

```
<CheckBox
    android:id="@+id/simpleCheckBox"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Padding Attribute Of Check Box"
    android:textColor="#44f"
    android:textSize="20sp"
    android:textStyle="bold|italic"
```

```
android:checked="false"
android:padding="30dp"/> <!--30dp padding from all side's-->
```

Android Spinner

Android spinner is like the drop down menu with multiple values from which the end user can select only one value.

If we are going to display the country list. You need to use **ArrayAdapter** class to store the country list.

```
1. <Spinner
2.     android:id="@+id/spinner"
3.     android:layout_width="149dp"
4.     android:layout_height="40dp"
5.     android:layout_marginBottom="8dp"
6.     android:layout_marginEnd="8dp"
7.     android:layout_marginStart="8dp"
8.     android:layout_marginTop="8dp"
9.     app:layout_constraintBottom_toBottomOf="parent"
   app:layout_constraintEnd_toEndOf="parent"
   app:layout_constraintHorizontal_bias="0.502"
   app:layout_constraintStart_toStartOf="parent"
   app:layout_constraintTop_toTopOf="parent"
   app:layout_constraintVertical_bias="0.498" />
```

File: MainActivity.java

```
1. package example.com.spinner;
2.
3. import android.support.v7.app.AppCompatActivity;
4. import android.os.Bundle;
5. import android.view.View;
6. import android.widget.AdapterView;
7. import android.widget.ArrayAdapter;
8. import android.widget.Spinner;
9. import android.widget.Toast;
```

```
public class MainActivity extends AppCompatActivity implements
```

```
    AdapterView.OnItemSelectedListener {
    String[] country = { "India", "USA", "China", "Japan", "Other"};
```

```
@Override
```

```
protected void onCreate(Bundle savedInstanceState) {
```

```
    super.onCreate(savedInstanceState);
```

```
    setContentView(R.layout.activity_main);
```

```
    //Getting the instance of Spinner and applying OnItemSelectedListener on it
```

```
Spinner spin = (Spinner) findViewById(R.id.spinner);  
spin.setOnItemSelectedListener(this);
```

```
//Creating the ArrayAdapter instance having the country list
```

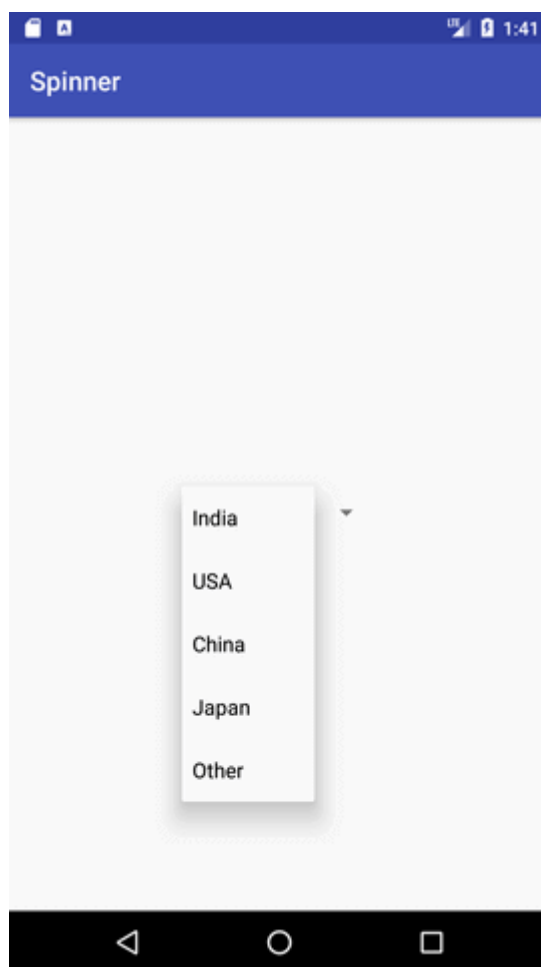
```
ArrayAdapter aa = new ArrayAdapter(this,android.R.layout.simple_spinner_item,c  
ountry);  
aa.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);  
//Setting the ArrayAdapter data on the Spinner  
spin.setAdapter(aa);
```

```
}
```

```
//Performing action onItemSelected and onNothing selected
```

```
@Override
```

```
public void onItemSelected(AdapterView<?> arg0, View arg1, int position, long id) {  
    Toast.makeText(getApplicationContext(),country[position] , Toast.LENGTH_LONG).  
show();  
}
```



Imageview

ImageView class is used to display any kind of image resource in the android application either it can be **android.graphics.Bitmap** or **android.graphics.drawable**.

XML Attributes of ImageView

XML Attribute	Description
android:id	To uniquely identify an image view
android:src/app:srcCompat	To add the file path of the inserted image
android:background	To provide a background color to the inserted image
android:layout_width	To set the width of the image
android:layout_height	To set the height of the image
android:padding	To add padding to the image from the left, right, top, or bottom of the view
android:scaleType	To re-size the image or to move it in order to fix its size

Android Image Switcher

Android image switcher provides an animation over images to transition from one image to another. In order to use image switcher, we need to implement **ImageSwitcher** component in .xml file.

The *setFactory()* method of ImageSwitcher provide implementation of *ViewFactory* interface.

Attributes:

```
<ImageSwitcher
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/imageSwitcher"
    android:layout_below="@+id/textView"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="168dp" />
```

Event Handling

Events are the actions performed by the user in order to interact with the application, for e.g. pressing a button or touching the screen. The events are managed by the android framework in the **FIFO** manner i.e. First In – First Out. Handling such actions or events by performing the desired task is called **Event Handling**.

Overview of the input event management

- **Event Listeners:** It is an interface in the View class. It contains a single callback method. Once the view to which the listener is associated is triggered due to user interaction, the callback methods are called.
- **Event Handlers:** It is responsible for dealing with the event that the event listeners registered for and performing the desired action for that respective event.

Event Listeners and their respective event handlers

- **OnClickListener()** – This method is called when the user clicks, touches, or focuses on any view (widget) like Button, ImageButton, Image, etc. **Event handler used for this is onClick().**
- **OnLongClickListener()** – This method is called when the user presses and holds a particular widget for one or more seconds. **Event handler used for this is onLongClick().**
- **OnMenuItemClickListener()** – This method is called when the user selects a menu item. **Event handler used for this is onMenuItemClick().**

- **OnTouch()** – This method is called either for a movement gesture on the screen or a press and release of an on-screen key. **Event handler used for this is onTouch()**.

Android Toast

Android Toast can be used to display information for the short period of time. A toast contains message to be displayed quickly and disappears after sometime.

Constants of Toast class

There are only 2 constants of Toast class which are given below.

Constant	Description
public static final int LENGTH_LONG	displays view for the long duration of time.
public static final int LENGTH_SHORT	displays view for the short duration of time.

Methods of Toast class

The widely used methods of Toast class are given below.

Method	Description
public static Toast makeText(Context context, CharSequence text, int duration)	makes the toast containing text and duration.
public void show()	displays toast.
public void setMargin (float horizontalMargin, float verticalMargin)	changes the horizontal and vertical margin difference.